ChainOptNet: An intelligent multi-level learning framework for adaptive blockchain system performance optimization

ARTICLE INFO

Keywords:
Blockchain system
Performance optimization
Transaction arrival rate prediction
Deep reinforcement learning
Machine learning
Block size adjustment

ABSTRACT

Blockchain has emerged as a transformative technology with numerous applications due to its tamper-proof, traceable, and decentralized architecture. Nonetheless, its performance remains constrained, particularly in areas such as latency and throughput, which pose significant barriers to large-scale deployment. This paper proposes ChainOptNet, a novel blockchain system performance optimization framework that addresses these challenges with three key technical contributions. First, we develop the Convolutional Feature Aggregation and Distribution (CFAD) model to forecast transaction arrival rates. This model uses a deep learning architecture to capture transaction data's temporal dependencies and complex statistical patterns. Second, we present a multi-model collaborative framework for blockchain performance prediction that improves predictive accuracy and robustness by intelligently integrating diverse machine learning models. Third, we create a Dynamic Weight Deep Q-Network (DW-DQN) optimization mechanism that combines a DQN-based block-size controller with a trend-aware dynamic reward weighting module for balanced multi-objective optimization. Extensive experimental evaluations demonstrate that ChainOptNet significantly improves both latency and throughput across a wide range of load conditions, consistently outperforming representative forecasting, regression, ensemble-learning, and score-based optimization baselines. Across the evaluated workload settings, ChainOptNet reduces average transaction latency by up to 21.4% and improves throughput by up to 18.7% compared with the best-performing baseline among the evaluated categories. These results demonstrate the framework's scalability, adaptability, and strong potential for practical deployment in diverse blockchain environments.

1. Introduction

Blockchain technology has quickly evolved into a transformative paradigm across multiple domains, providing decentralized, tamper-resistant, and transparent solutions for data exchange and transaction processing [1],[2],[3],[4]. Its fundamental properties, which include immutability, traceability [5],[6], and consensus-driven trust [7], have enabled widespread adoption in fields such as digital finance [8],[9], supply chain management [10],[11], healthcare [12], the Internet of Things (IoT) [13],[14],[15],[16], and smart grids [17],[18],[19],[20].

Despite these benefits, blockchain systems still face significant performance bottlenecks, particularly in terms of scalability [21],[22],[23]. Existing public blockchains, such as Bitcoin and Ethereum, have limited transaction processing capabilities, averaging around 7 and 30 transactions per second (TPS), respectively [24],[25]. These rates fall far short of the requirements for high-throughput applications, such as financial exchanges and IoT networks [26],[27],[28]. In addition, the latency required for transaction confirmation can range from a few seconds to minutes, making blockchain unsuitable for latency-sensitive services like real-time payments or industrial automation [29],[30],[31]. These limitations are exacerbated by dynamic workloads, in which transaction arrival rates and network conditions (such as node participation and network latency) fluctuate unpredictably as a result of external user behavior or system-level events. In such cases, traditional blockchain configurations, which are statically defined at the time of deployment, lack the responsiveness required to adapt to real-time changes. This causes to increased block propagation delays, transaction congestion, and inefficient resource utilization. Moreover, practical blockchain deployments are often affected by fluctuating node participation, heterogeneous hardware capabilities, and geographically distributed network infrastructures. These factors introduce varying communication delays and resource availability, further complicating performance optimization. Therefore, an effective optimization framework should continuously adapt to workload evolution while maintaining stable performance under changing network conditions.

To address these performance issues, researchers have proposed a number of improvement strategies. Alternative consensus algorithms such as Proof-of-Stake (PoS) and Practical Byzantine Fault Tolerance (PBFT), aim to reduce consensus latency and increase throughput [32],[33],[34],[35], but they frequently compromise decentralization and scalability. Some approaches use heuristic or rule-based tuning of system parameters, such as adjusting block size or gas limits based on network statistics [36],[37],[38],[39],[40]. Some researchers have also introduced machine learning-based methods for tuning blockchain parameters [41], with the goal of optimizing performance in a more adaptive and data-driven manner. Recently, some researchers have used performance scoring mechanisms to dynamically optimize blockchain configuration parameters [42]. These methods can effectively supplement and synergize with existing efforts, providing a promising path to improving blockchain performance.

However, these strategies typically lack adaptability and robustness because they rely on manually defined thresholds that may be insufficient under unpredictable or changing conditions. Furthermore, while machine learning methods

ORCID(s):

provide more flexibility, the accuracy of the proposed performance prediction models can still be improved, and their scalability is frequently limited by high training data requirements, which may result in additional data collection and computational costs. Furthermore, while advances such as improved consensus algorithms and blockchain sharding have demonstrated potential in controlled environments [32],[43], their practical implementation frequently encounters significant technical barriers and intricate customization requirements. These include the varying complexity of real-world application scenarios and the diverse needs of blockchain users, which limit the solutions' universal applicability. Notably, recent scoring-based optimization methods [42] also face challenges such as static weight configurations, limited real-time adaptability, manually defined tuning rules, limited deployment scope, and inadequate handling of multi-objective trade-offs. These limitations reduce their effectiveness in dynamic and heterogeneous blockchain environments. Overall, most existing methods are constrained by rigid architectures or fixed policies, which significantly reduces their effectiveness in rapidly changing operational environments.

To address the aforementioned limitations, we introduce ChainOptNet, a unified and adaptive blockchain performance optimization framework that integrates key processes, such as transaction arrival rate prediction, performance forecasting, and dynamic parameter tuning. In contrast to existing methods that handle these tasks in isolation, ChainOptNet uses a tightly coupled design that allows for real-time adaptation to dynamic network environments. The framework achieves high responsiveness and robustness by systematically coordinating learning-based components. Extensive evaluations across a variety of blockchain scenarios show consistent improvements in throughput and latency over representative baselines from forecasting, regression, ensemble-learning, and score-based optimization categories, highlighting its strong adaptability and scalability in complex environments. The primary contributions of this paper are summarized as follows:

- We introduce CFAD, a convolutional sequence model designed to capture complex nonlinear temporal dependencies found in blockchain transaction patterns. CFAD allows for accurate short-term and mid-term forecasting of transaction arrival rates, which serves as a crucial input for downstream optimization.
- We propose a collaborative multi-model performance prediction framework that includes heterogeneous learning models. The framework improves forecasting robustness across different performance indicators by using inverse error-weighted ensemble learning, even when the historical data is noisy or incomplete.
- We present DW-DQN, a reinforcement learning-based optimization module with a trend-aware dynamic reward weighting mechanism. The DQN learns the block-size adjustment policy, while the reward weights for latency and throughput are deterministically computed from recent performance trends, enabling efficient multi-objective trade-offs in dynamic network environments.
- We run extensive experiments across a wide range of blockchain scenarios to validate ChainOptNet's superior performance, adaptability, and scalability under varying workloads and network conditions. The findings show that ChainOptNet can learn effective parameter adjustment strategies by leveraging temporal patterns, ensemble learning, and dynamic reward tuning, demonstrating ChainOptNet's effectiveness and superiority for blockchain system performance optimization.

The remainder of this paper is organized as follows. Section 2 reviews related work on blockchain performance prediction and optimization. Section 3 presents the motivation for this study. Section 4 presents the architecture and operational design of ChainOptNet. Section 5 details the experimental setup and presents comparative analyses. Section 6 concludes the paper and discusses future research directions.

2. Related work

As blockchain technology becomes increasingly widespread across various applications, performance issues such as low throughput and high latency have emerged as major bottlenecks. Existing research has explored a range of optimization strategies from different perspectives to address these challenges.

2.1. Improvement of consensus algorithms

Improving the consensus mechanism is one of the most common approaches to enhancing blockchain performance. Gao et al. [33] proposed a piecewise hash graph consensus algorithm designed for industrial IoT, which significantly improves throughput and reduces latency. Zhang et al. [34] enhanced the performance of Hyperledger Fabric by modifying its consensus module, thereby achieving a better trade-off between security and efficiency. Xu et al. [32] introduced AdaptChain, an elastic blockchain that supports adaptive expansion and contraction based on transaction volume, optimizing resource utilization while maintaining high performance. Li et al. [44] proposed a hybrid consensus mechanism that integrates intra-shard processing with lightweight global coordination, addressing the issues of vulnerable consensus security and delayed atomicity in cross-shard transactions while maintaining adaptive shard management. Islam et al. [45] proposed Proof of Random Leader (PoRL), a PoA-based consensus algorithm that employs verifiable random functions to enhance throughput and ensure fair leader selection, thus addressing the limitations of low throughput and predictable leader rotation in existing PoA variants such as Aura and Clique.

2.2. Blockchain sharding

Sharding techniques partition the blockchain network into multiple shards to enhance parallelism and scalability. Cai et al. [46] developed Benzene, a collaboration-based sharding system that improves throughput while maintaining shard-level security. Li et al. [43] presented LBChain, a load-balancing sharding scheme that dynamically migrates accounts among shards based on activity levels, effectively balancing transaction loads and boosting throughput. Yang et al. [47] introduced EZchain, a passive sharding protocol that achieves constant communication and storage costs without requiring cross-shard transactions or beacon chains, offering improved scalability and security in large networks. Liu et al. [48] proposed a flexible sharding (FS) blockchain protocol that features cross-shard Byzantine fault tolerance (CSBFT) to reduce confirmation delays, multiple parallel CSBFT to enhance system parallelism, and decentralized shard reconfiguration. This effectively addresses key challenges in cross-shard transaction processing and security vulnerabilities. Jia et al. [49] proposed a low cross-shard protocol, which features a multi-level state model and a community overlap propagation algorithm to dramatically reduce cross-shard transactions. The X-shard system [50] addresses cross-shard processing bottlenecks through transaction pattern-based allocation and optimistic parallel processing, achieving near-linear throughput scaling and reduced latency.

2.3. Optimization based on traditional algorithms

Several studies have employed classical optimization algorithms to fine-tune blockchain parameters. Jamil et al. [38] applied Particle Swarm Optimization (PSO) to evaluate the trade-off between throughput and latency in Hyperledger Fabric, aiming to determine the optimal transaction arrival rate. Chen et al. [39] deployed a private Ethereum IoT network and used probabilistic fitting to model its latency performance. Wilhelmi et al. [40] utilized Markov Chains to determine the optimal block size configuration for minimizing latency through numerical analysis.

2.4. Machine learning-based optimization

Recent advances have introduced machine learning (ML) to predict system behavior and support intelligent decision-making in blockchain environments. Wang et al. [41] proposed a performance prediction framework that employs Multilayer Perceptron (MLP) and Long Short-Term Memory (LSTM) networks to predict blockchain performance and transaction arrival rates. However, this approach does not address issues such as the cost of collecting blockchain performance data, its generalization capability under small sample sizes, and the integration of predictive modeling with system-level optimization. Other prediction-driven optimization methods combine temporal workload forecasting, performance prediction, and score-based configuration adjustment to improve throughput and latency [42]. Although these methods provide a more flexible alternative to manual tuning, they often depend on predefined scoring functions or fixed decision rules, making it difficult to adapt the optimization objective when latency and throughput priorities change across operating regimes.

3. Research motivation

This section identifies key limitations in current blockchain optimization: the disjoint handling of workload prediction and system tuning, the poor generalization of machine learning models under data scarcity and noise, and the lack of adaptability and real-time responsiveness in dynamic environments.

3.1. Disjoint optimization and lack of adaptability in dynamic environments

Current blockchain optimization strategies often treat transaction workload prediction and system-level configuration separately, and many of them use static configurations or rule-based policies, assuming stable environments [38],[39],[40],[41],[42],[51]. In dynamic blockchain environments, transaction arrival rates fluctuate unpredictably due to user behavior or external events, and node participation and transaction volumes are also volatile. Without tight integration between prediction and control, and without adaptive strategies that can autonomously update configurations in response to observed changes, workload predictions cannot effectively guide proactive adjustments to system parameters like block size or gas limit, leading to inefficient resource allocation, increased latency, and poor performance under shifting workloads.

3.2. Limited generalization under data constraints

Machine learning–based blockchain performance prediction often relies on abundant, high-quality training data, which is rarely available in real-world deployments. Public blockchains may have imbalanced datasets, while consortium blockchains often suffer from small sample sizes [52]. Additionally, blockchain data is frequently noisy due to inconsistent node behaviors or heterogeneous network conditions, challenging learning algorithms and limiting model generalization [53]. A robust framework must incorporate models resilient to such data deficiencies.

3.3. Summary

To address these gaps, this paper proposes ChainOptNet, a unified blockchain performance optimization framework featuring: (i) a deep learning–based transaction arrival rate prediction model (CFAD), (ii) a multi-model ensemble for performance prediction, and (iii) a dynamic weight reinforcement learning strategy (DW-DQN) for real-time block size adjustment.

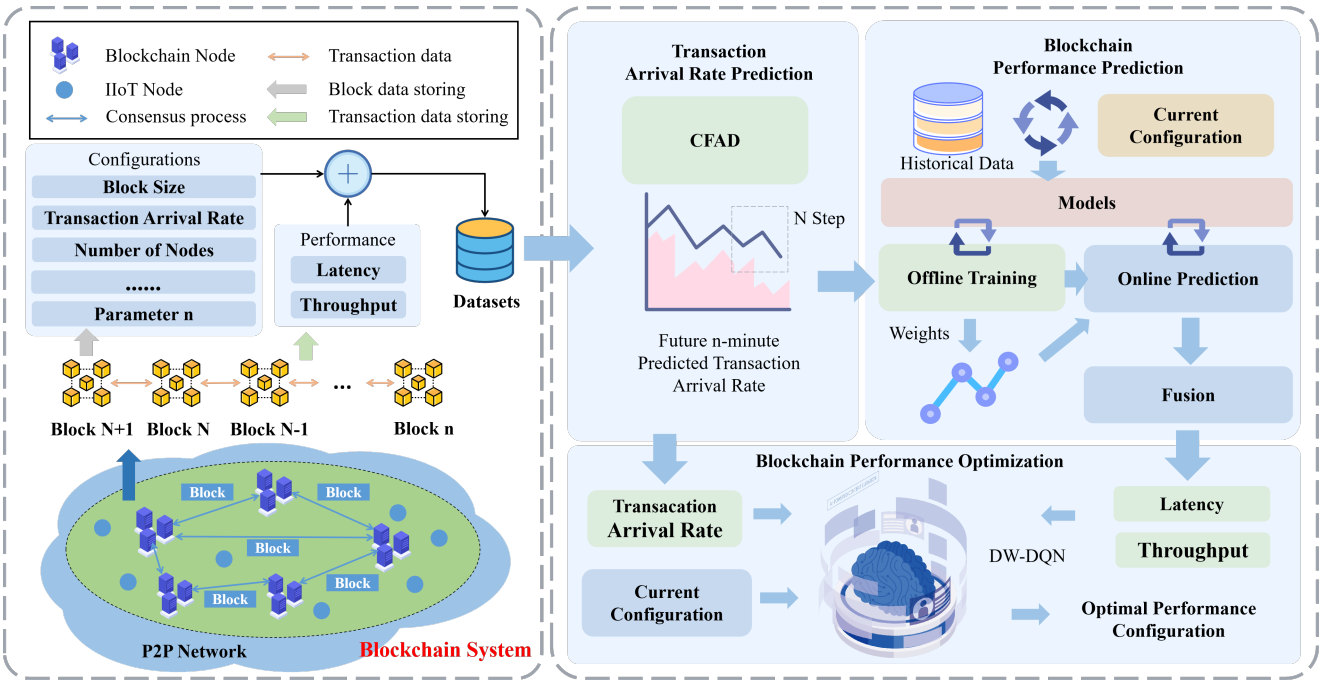


Figure 1: The framework of ChainOptNet.

4. System model

4.1. Overall framework

Figure 1 depicts the proposed ChainOptNet framework for blockchain performance optimization, which dynamically adjusts the block size according to real-time network conditions and historical transaction information. The framework takes historical transaction arrival rates together with blockchain configuration parameters, including block size, the number of ordering nodes, latency, and throughput, as inputs, and outputs the optimal block size to improve overall blockchain performance. As illustrated in Fig. 1, the proposed framework consists of the following three components:

(1) Transaction arrival rate prediction: The Convolutional Feature Aggregation and Distribution (CFAD) model predicts future transaction arrival rates from historical transaction data by effectively capturing temporal dependencies and local fluctuation patterns.

(2) Blockchain performance prediction: A multi-model collaborative prediction framework integrates seven machine learning models, including Linear Regression, Bayesian Ridge Regression, Decision Tree Regressor, Support Vector Regression, Gradient Boosting Regressor, Random Forest Regressor, and eXtreme Gradient Boosting Regressor, to estimate blockchain performance metrics such as latency and throughput. The prediction results are further fused using an inverse error-weighted ensemble strategy, thereby improving prediction robustness and accuracy.

(3) Blockchain performance optimization: A Dynamic Weight Deep Q-Network (DW-DQN) dynamically determines the optimal block size. Specifically, the optimization module consists of a DQN-based block-size controller and a trend-aware reward weighting module. The DQN learns the optimal block-size adjustment policy, while the reward weighting module adaptively computes the relative importance of latency and throughput according to recent performance trends, enabling adaptive multi-objective optimization.

4.2. Prediction of transaction arrival rate

Inspired by the recently proposed SOFTS model [54], which demonstrates efficient and scalable performance in multivariate time series forecasting via its centralized STAR (STar Aggregate-Redistribute) module, we create a modified version specifically for predicting blockchain transaction arrival rates. Unlike SOFTS, which focuses on general multivariate forecasting across a variety of domains, our approach adapts and simplifies the centralized interaction mechanism by incorporating convolutional operations for both temporal and channel-wise modeling, making it better suited to the characteristics of blockchain transaction data. This leads to the creation of the Convolutional Feature Aggregation and Distribution (CFAD) framework, which preserves SOFTS' low complexity advantage while improving its suitability for blockchain-specific scenarios.

To address the challenges of blockchain performance optimization, particularly in accurately predicting transaction arrival rates, we propose the Convolutional Feature Aggregation and Distribution (CFAD) framework. This framework is intended to forecast multivariate transaction arrival

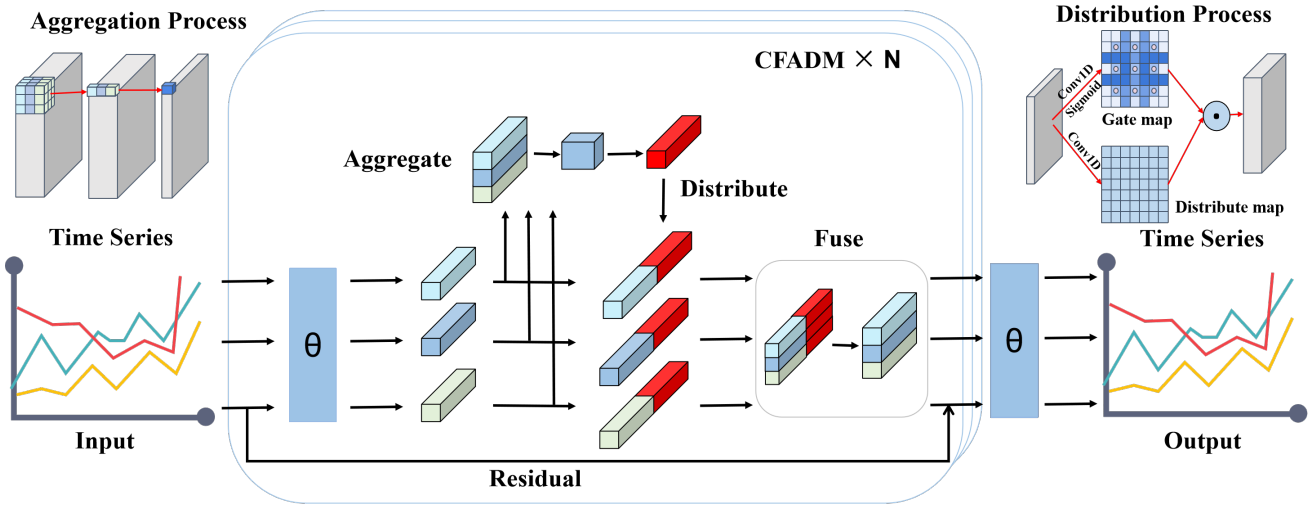


Figure 2: The framework of CFAD.

rates efficiently, with less computational complexity than traditional methods. This method allows the model to capture both spatial and temporal dependencies in blockchain transaction data, which improves prediction accuracy and system performance. As shown in Figure 2, the CFAD framework consists of three main components: sequence embedding, the Convolutional Feature Aggregation and Distribution Module (CFADM), and convolutional output projection.

4.2.1. Sequence embedding

The sequence embedding component of CFAD treats the entire transaction time series as a single "patch" processed through a 1D convolutional layer. This approach avoids the dimensionality expansion typically caused by patch embedding methods and preserves the temporal relationships in the data. By using a convolutional layer with a kernel size of 1 and 'same' padding, CFAD efficiently preserves the temporal relationships without dimensionality expansion:

$$S_0 = \text{Conv1D}_{\text{kernel}=1}(X), \quad X \in \mathbb{R}^{L \times C} \tag{1}$$

where X represents the input transaction data with L time steps and C features (channels).

4.2.2. Convolutional feature aggregation and distribution module (CFADM)

The Convolutional Feature Aggregation and Distribution Module (CFADM) is CFAD's core component, which aggregates and distributes features from blockchain transaction data. Formally,

$$S_N = \text{CFADM}(S_0) \tag{2}$$

where S_N denotes the output of the N-th CFADM layer. The module has two phases:

Aggregation Phase: During this phase, dual-layer 3×3 convolutions with 'same' padding and ReLU activation are used to aggregate temporal features from the input sequence. The aggregation process enables the model to learn implicit temporal dependencies throughout the sequence, resulting in a rich global feature representation of the transaction data:

$$S_i^{agg} = \text{Conv1D}_{\text{aggregate, kernel}=3}(S_{i-1}) \tag{3}$$

$$S_i^{agg} = \text{ReLU}(S_i^{agg}), \quad i = 1, 2, \ldots, N \tag{4}$$

where S_i^{agg} represents the aggregated feature at the i-th layer.

Distribution Phase: To enhance the preservation of layer-wise input information, we concatenate the aggregated features with the pre-aggregation input S_{i-1} before distribution, forming a richer feature representation for subsequent gating and distribution. Specifically, features are first concatenated along the channel dimension:

$$S_i^{concat} = \text{Concat}(S_i^{agg}, S_{i-1}) \tag{5}$$

where Concat$(\cdot)$ denotes concatenation along the channel dimension, allowing the model to integrate both pre-aggregation input features and aggregated features.

The concatenated features are then redistributed across channels using two 1×1 convolutional branches combined with a gating mechanism. The gating is implemented via a sigmoid activation to control the flow of information, allowing the model to dynamically adjust the importance of each feature:

$$G_i = \sigma(\text{Conv1D}_{\text{gate}}(S_i^{concat})) \tag{6}$$

$$D_i = \text{Conv1D}_{\text{distribute, kernel}=1}(S_i^{concat}) \tag{7}$$

$$S_i^{out} = G_i \odot D_i \tag{8}$$

where $\sigma(\cdot)$ denotes the sigmoid activation function for gating, and $\odot$ denotes element-wise multiplication. This step ensures that the most relevant features, enriched by both the current layer input and the aggregated global context, are distributed to subsequent layers for accurate prediction.

4.2.3. Convolutional output projection

Let S_N represent the output of the N-th CFADM layer, which encodes aggregated and distributed temporal features. After the CFADM layers, the final feature representation is passed through a 1×1 convolutional layer to project the aggregated and distributed features into the predicted transaction arrival rates. Compared to a traditional fully connected layer, this convolutional output layer preserves the sequence's temporal structure and allows for localized prediction, which is especially useful for multi-step predicting:

$$Y = \text{Conv1D}_{\text{out, kernel}=1}(S_N) \tag{9}$$

where $Y \in \mathbb{R}^{L' \times 1}$ represents the predicted transaction arrival rates for future L' time steps.

4.3. Blockchain performance prediction model

To address the limitations of single machine learning models in capturing complex interactions among features, this study proposes a multi-model collaborative prediction framework that combines seven different machine learning algorithms. This framework enhances predictive accuracy and adaptability in volatile blockchain environments by leveraging model diversity and dynamic weighting.

The proposed framework integrates the following base regressors: Linear Regression (LR), Bayesian Ridge Regression (BYR), Decision Tree Regressor (DTR), Bagging Regressor (BR), Gradient Boosting Regressor (GBR), Random Forest Regressor (RFR), and eXtreme Gradient Boosting (XGB). These models cover both parametric and non-parametric domains, allowing the ensemble to capture a wide range of data characteristics, including linear dependencies and complex nonlinear patterns.

Figure 3 depicts the complete workflow for the proposed multi-model framework. It is divided into two major stages: (i) an offline training phase, in which base learners are trained on enriched features and weighted using inverse-error-based softmax; and (ii) an online prediction phase, in which model weights are dynamically updated using a sliding window of recent observations to maintain adaptability under changing conditions.

Let the input feature vector be denoted as $X = [x_1, x_2, \cdots, x_n] \in \mathbb{R}^d$, comprising normalized system parameters such as transaction arrival rate, block size, and number of ordering nodes. The corresponding target vector $Y \in \mathbb{R}^k$ represents performance indicators including latency and throughput.

Prior to model training, the input features are standardized using z-score normalization:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}, \tag{10}$$

where μ and σ denote the mean and standard deviation of each feature, respectively.

To enrich the feature representation, second-order interactions are explicitly computed as follows:

$$X_{\text{interaction}} = [X \otimes X], \tag{11}$$

where $\otimes$ denotes the outer product of the feature vector. The resulting interaction terms are concatenated with the normalized input to form an extended feature vector:

$$X_{\text{extended}} = [X_{\text{scaled}}, X_{\text{interaction}}]. \tag{12}$$

Each base model $f_m(\cdot)$, parameterized by θ_m, produces an individual prediction $\hat{Y}_m$ as follows:

$$\hat{Y}_m = f_m(X_{\text{extended}}; \theta_m). \tag{13}$$

To evaluate model performance, the Mean Absolute Error (MAE) is adopted. Let the total number of samples be N, and assume each sample's true label is $Y_i \in \mathbb{R}^k$, the MAE of model m over the dataset is computed as follows:

$$\text{MAE}_m = \frac{1}{N} \sum_{i=1}^{N} \left\| \hat{Y}_{i,m} - Y_i \right\|_1, \tag{14}$$

where $\| \cdot \|_1$ denotes the element-wise L_1-norm across the performance metrics.

Based on these MAE scores, an inverse-error-based softmax function determines the ensemble weights:

$$W_m = \frac{\exp(-\text{MAE}_m / T)}{\sum_{j=1}^{M} \exp(-\text{MAE}_j / T)}, \tag{15}$$

where T is a temperature hyperparameter controlling the sharpness of the distribution.

In the online prediction phase, model performance is continuously reassessed using a sliding window of the most recent N samples. At time step t, let the windowed data be $\{(X_{t-i}, Y_{t-i})\}_{i=0}^{N-1}$. The updated prediction error for model m is calculated as follows:

$$\text{MAE}_m^{(t)} = \frac{1}{N} \sum_{i=0}^{N-1} \left\| f_m(X_{t-i}; \theta_m) - Y_{t-i} \right\|_1. \tag{16}$$

The corresponding dynamic weight is:

$$W_m^{(t)} = \frac{\exp(-\text{MAE}_m^{(t)} / T)}{\sum_{j=1}^{M} \exp(-\text{MAE}_j^{(t)} / T)}. \tag{17}$$

and the ensemble prediction becomes:

$$\hat{Y}^{(t)} = \sum_{m=1}^{M} W_m^{(t)} \cdot \hat{Y}_m^{(t)}. \tag{18}$$

While $\hat{Y}^{(t)}$ serves as the final output of the ensemble during the inference phase, the training process aims to optimize the base model parameters $\{\theta_m\}_{m=1}^{M}$ by minimizing

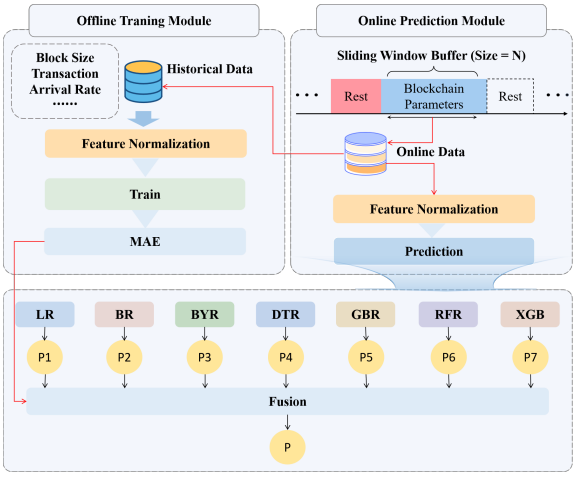


Figure 3: The framework of multi-model performance prediction.

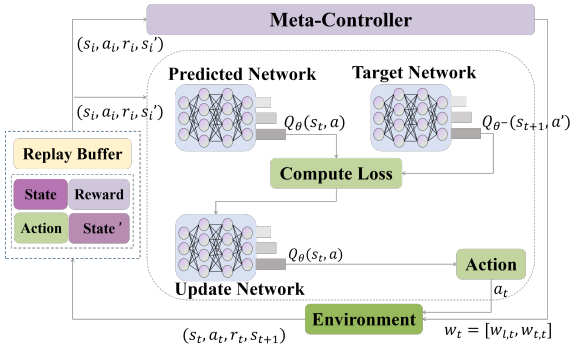


Figure 4: The framework of DW-DQN.

a composite loss function that reflects the accuracy of both latency and throughput predictions.

To ensure balanced learning across both objectives, the training process minimizes the following composite loss function:

$$L = \alpha L_{\text{latency}} + (1-\alpha) L_{\text{throughput}}, \tag{19}$$

where $\alpha \in [0, 1]$ is a hyperparameter controlling the relative importance of latency and throughput prediction. Cross-validation using grid search determines the optimal α value.

4.4. Blockchain performance optimization with DW-DQN

The proposed Dynamic Weight Deep Q-Network (DW-DQN) framework is intended to address the limitations of traditional reinforcement learning approaches in multi-objective optimization tasks within blockchain environments, particularly in the dynamic adjustment of block size to optimize throughput and latency. Unlike traditional Deep Q-Networks (DQNs), which use a static reward formulation, DW-DQN combines a DQN-based block-size controller with a trend-aware dynamic reward weighting module. The DQN learns the action policy, while the weighting module deterministically updates the latency and throughput weights according to recent system performance trends.

Figure 4 depicts the architecture and interconnections of the system's components, providing a comprehensive overview of the proposed DW-DQN framework. The framework is made up of two coordinated components: (1) a DQN agent that learns an optimal block-size adjustment policy from environmental interactions, and (2) a trend-aware reward weighting module that computes the weight vector used in reward computation. The state information is used by the DQN for action selection and by the weighting module for reward prioritization.

Formally, the learning process is modeled as a Markov Decision Process (MDP) $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ represents the action space, $\mathcal{P}$ denotes the state transition probabilities, $\mathcal{R}$ represents the reward function, and $\gamma \in [0, 1)$ denotes the discount factor. At time step t, the system state is represented as follows:

$$s_t = [\lambda_t, b_t, n_t, l_t, \tau_t] \tag{20}$$

where λ_t denotes the transaction arrival rate, b_t represents the current block size, n_t denotes the number of ordering nodes, l_t represents the average latency, and τ_t denotes the average throughput. The action a_t corresponds to a discrete change in block size $a_t \in \{-\Delta b, 0, +\Delta b\}$, where Δb is the step size.

Instead of a fixed scalar reward, the DW-DQN framework utilizes a dynamically weighted reward function defined as follows:

$$r_t = \omega_{l,t} r_t^{(l)} + \omega_{\tau,t} r_t^{(\tau)} \tag{21}$$

$$r_t^{(l)} = -|l_t - l^*| \tag{22}$$

$$r_t^{(\tau)} = \tau_t / \tau^* \tag{23}$$

where $r_t^{(l)}$ penalizes latency deviation from a target latency l^*, and $r_t^{(\tau)}$ reflects normalized throughput performance against a target τ^*. The weights $\omega_{l,t}$ and $\omega_{\tau,t}$ are dynamically computed at each time step by the trend-aware weighting module, subject to the constraint $\omega_{l,t} + \omega_{\tau,t} = 1$. These weights are not predicted by an additional neural network. Instead, they are derived from recent latency and throughput trends so that the reward function can emphasize the objective that is currently changing more sharply.

The DQN agent approximates the action-value function $Q_\theta(s_t, a_t)$, using a neural network with parameters θ. The network is trained to minimize the temporal-difference loss:

$$L(\theta) = \mathbb{E}_{\mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a) \right)^2 \right] \tag{24}$$

where θ^- are the parameters of the target network Q_{θ^-}, which are periodically updated from θ, and $\mathcal{D}$ denotes the replay buffer that stores transitions (s_t, a_t, r_t, s_{t+1}).

The trend-aware weighting module maintains recent latency and throughput observations and computes their trends via first-order differences. A normalized ratio is then used to determine the relative importance of latency and throughput.

$$\text{latency_importance} = \frac{|\text{trend}_l|}{|\text{trend}_l| + |\text{trend}_\tau| + \varepsilon} \tag{25}$$

$$\text{throughput_importance} = 1 - \text{latency_importance} \tag{26}$$

where trend_l and trend_τ denote the mean first-order differences of recent latency and throughput, respectively, and ε is a small constant to avoid division by zero. These importance scores are passed through a softmax layer to ensure probabilistic normalization. The resulting weights $\omega_{l,t}$ and $\omega_{\tau,t}$ are then clipped to lie within a specified range $[\omega_{\min}, \omega_{\max}]$, and re-normalized to enforce $\omega_{l,t} + \omega_{\tau,t} = 1$.

The objective of the DQN agent is to maximize the expected cumulative dynamically weighted reward:

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \tag{27}$$

where R_t denotes the cumulative discounted reward starting at time step t, based on the dynamically weighted reward formulation. Only the Q-network parameters are learned through temporal-difference updates; the reward weights are computed by the deterministic trend-aware module and are not updated by a separate learned policy.

This design allows the action policy to be learned by DQN while the reward priorities are adapted online according to observed network behavior. The DW-DQN framework demonstrates robust adaptability by dynamically modulating reward priorities through a deterministic trend-aware rule, significantly improving blockchain system performance under a variety of operational scenarios.

5. Experimental evaluation and analysis

This section contains a series of experiments designed to assess the performance and effectiveness of the proposed methods. Qualitative comparisons with related work are also provided to highlight the novelty and benefits.

5.1. Experimental settings

5.1.1. Experimental design

We conducted three sets of experimentss: (1) predicting transaction arrival rates, (2) predicting blockchain performance through inverse error ensemble learning, and (3) optimizing blockchain performance.

5.1.2. Datasets

In Experiment 1 used an open-source dataset of parking lot records collected in real-world scenarios to predict transaction arrival rates. Although originally designed for parking behavior analysis, the dataset's realistic temporal characteristics make it suitable for modeling blockchain transaction arrivals.

For Experiments 2 and 3, we used the HFBDP dataset, an open-source blockchain performance dataset compiled by [42] using Hyperledger Fabric. The dataset has five key parameters: orderer node count (3, 5, 7, 9), transaction arrival rate (10, 15, 20, …, 200), block size (10, 15, 20, …, 800), latency, and throughput. These parameters were applied to the blockchain performance prediction and optimization tasks, respectively.

5.1.3. Comparable methods

To comprehensively evaluate the effectiveness of the proposed ChainOptNet framework, representative baseline methods were selected for each task.

For transaction arrival rate prediction, we compared the proposed CFAD model with several representative time-series forecasting models, including Autoformer, Informer, Transformer, LSTM, GRU, LightTS, MLP, and a blockchain-oriented temporal prediction baseline [42]. These methods cover recurrent neural networks, Transformer-based architectures, lightweight forecasting models, and domain-specific prediction methods.

For blockchain performance prediction, the proposed multi-model collaborative prediction framework was compared with several widely used regression models, including Linear Regression (LR), Bayesian Ridge Regression (BR), Bayesian Regularization (BYR), Decision Tree Regression (DTR), Gradient Boosting Regression (GBR), Random Forest Regression (RFR), and eXtreme Gradient Boosting (XGB). These models represent linear regression, probabilistic regression, decision tree learning, and ensemble learning methods that are widely adopted for blockchain performance prediction.

For blockchain performance optimization, we compared the proposed DW-DQN with representative non-RL optimization settings, including fixed block-size configurations and a prediction-driven score-based adjustment strategy [42]. Such methods adjust parameters according to predefined performance scores or static configuration rules, whereas DW-DQN formulates block-size optimization as a reinforcement learning problem and dynamically adjusts reward weights according to network conditions, enabling more adaptive optimization under varying workloads.

To ensure fairness and reproducibility, all baseline methods were evaluated using identical datasets, preprocessing procedures, experimental settings, and evaluation metrics.

5.1.4. Evaluation metrics

To quantitatively evaluate the prediction and optimization performance of the proposed framework, three widely used regression metrics were adopted, namely Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE). During model training, the Mean Squared Error (MSE) was employed as

the optimization objective. These metrics provide a comprehensive evaluation of prediction accuracy, robustness, and generalization capability.

Mean Squared Error (MSE): During model training, the Mean Squared Error (MSE) was used as the loss function to optimize model parameters. It is defined as

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \tag{28}$$

where N denotes the number of samples, y_i is the ground-truth value, and $\hat{y}_i$ represents the corresponding prediction.

Mean Absolute Error (MAE): MAE measures the average magnitude of prediction errors regardless of their direction and is formulated as

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|. \tag{29}$$

A smaller MAE indicates higher prediction accuracy and better overall performance.

Root Mean Square Error (RMSE): RMSE measures the standard deviation of prediction errors and places greater emphasis on larger errors. It is calculated as

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}. \tag{30}$$

Lower RMSE values indicate better predictive performance and stronger robustness to large deviations.

Mean Absolute Percentage Error (MAPE): MAPE evaluates prediction accuracy in terms of relative error and is defined as

$$\mathrm{MAPE} = \frac{100\%}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right|. \tag{31}$$

MAPE is scale-independent and is particularly suitable for comparing prediction performance across different experimental settings.

5.2. The performance of ChainOptNet

5.2.1. Prediction performance on transaction arrival rate

To evaluate the generalization capability and robustness of the proposed Convolutional Feature Aggregation and Distribution (CFAD) model in predicting transaction arrival rates, experiments were conducted at three temporal granularities: 1-minute, 2-minute, and 5-minute intervals.

As shown in Figure 5 and Table 1, CFAD consistently outperforms several existing methods across all time scales

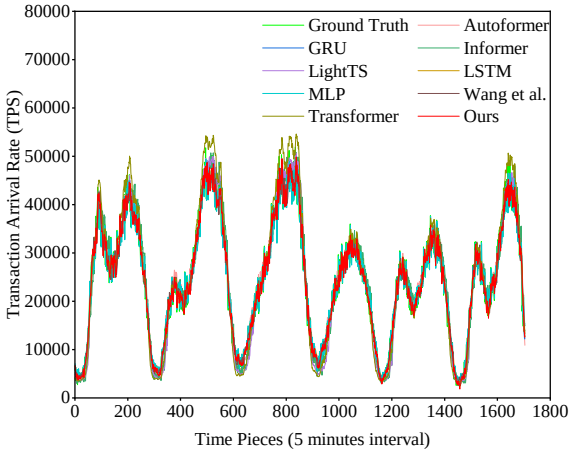


Figure 5: Prediction results of different models

tested. This superior performance is attributed to CFAD's global one-dimensional convolution-based sequence encoder, which preserves the inherent sequential structure of transaction data, allowing for better capture of fine-grained temporal dependencies and improved responsiveness to sudden fluctuations.

In the 5-minute prediction scenario, CFAD achieved significant improvements in MAE, RMSE, and MAPE compared with representative temporal forecasting and blockchain-oriented prediction baselines. CFAD also demonstrated strong robustness during high-variance periods, reducing error variance by 4.92% compared with the prior blockchain-oriented prediction baseline. This robustness is attributed to its multi-scale convolutional structure and integrated gating mechanism, which enhance predictive stability under non-stationary conditions.

5.2.2. Performance of the blockchain performance prediction model

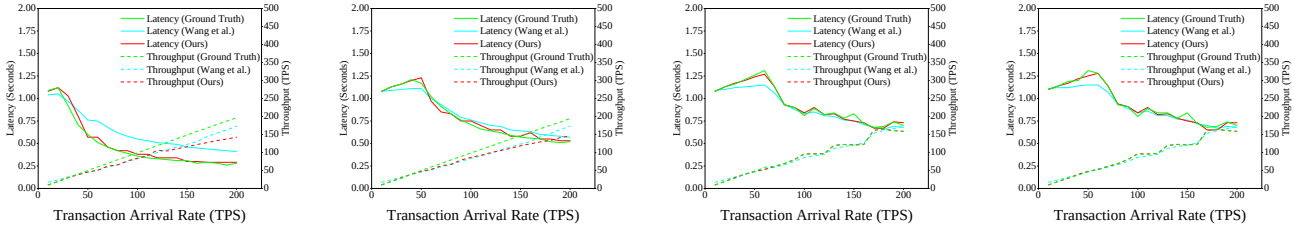
To evaluate the generalization capability and predictive performance of the proposed multi-model collaborative blockchain performance prediction framework, we conducted extensive experiments focusing on latency and throughput.

As shown in Figure 6 and Table 2, the proposed framework outperformed several categories of baseline methods, including traditional regression models (e.g., Linear Regression, Bayesian Ridge, Decision Tree Regression), ensemble-based learners (e.g., Gradient Boosting Regression, Random Forest Regression, XGBoost), and prior blockchain-oriented prediction methods [42]. Evaluations were conducted under various system configurations with block sizes of 50, 100, 150, and 200, and across a wide range of transaction arrival rates (TPS).

Latency prediction The proposed model achieved the lowest MAE (0.0223) and MAPE (0.0298) among the compared regression, ensemble-learning, and blockchain-oriented prediction baselines, as shown in Table 2. This demonstrates

Table 1
Prediction performance comparison across different time intervals.

Time periods	Metrics	Methods								
		Informer	LSTM	Transformer	GRU	MLP	AutoFormer	LightTS	Wang et al.	Ours
1 minute	MAE	0.0534	0.0404	0.0441	0.0405	0.0511	0.0430	0.0406	0.0406	**0.0400**
	RMSE	0.0658	0.0526	0.0583	0.0529	0.0662	0.0552	0.0528	0.0529	**0.0523**
	MAPE	0.2807	0.2861	0.2727	0.2849	0.3099	**0.2220**	0.2928	0.2699	0.2486
2 minutes	MAE	0.0564	0.0347	0.0432	0.0343	0.0419	0.0421	0.0401	0.0351	**0.0337**
	RMSE	0.0721	0.0455	0.0542	0.0449	0.0547	0.0540	0.0512	0.0461	**0.0441**
	MAPE	0.2232	0.1296	0.1772	0.1377	0.1713	0.1643	0.1580	0.1341	**0.1275**
5 minutes	MAE	0.0406	0.0282	0.0458	0.0277	0.0311	0.0324	0.0376	0.0293	**0.0247**
	RMSE	0.0510	0.0368	0.0601	0.0360	0.0404	0.0416	0.0479	0.0387	**0.0325**
	MAPE	0.1551	0.1145	0.1665	0.1121	0.1229	0.1225	0.1527	0.1085	**0.0887**



(a) Block size = 50 (b) Block size = 100 (c) Block size = 150 (d) Block size = 200

Figure 6: The latency and throughput prediction performance of the blockchain performance prediction model under different block sizes.

Table 2
Performance comparison on latency and throughput prediction tasks

Task	Method	MAE	RMSE	MAPE
Latency	LR	0.1237	0.1764	0.2256
	BR	0.2547	0.3242	0.4010
	BYR	0.2520	0.3166	0.3960
	DTR	0.0218	0.0597	0.4285
	GBR	0.2273	0.2905	0.3662
	RFR	0.2700	0.3590	0.4280
	XGB	0.2308	0.2945	0.3705
	Wang et al.	0.0253	**0.0481**	0.0408
	Ours	**0.0223**	0.0519	**0.0298**
Throughput	LR	0.2095	0.2699	0.0559
	BR	0.7303	0.9448	0.1896
	BYR	0.7705	0.9593	0.1972
	DTR	0.0164	0.0443	0.1962
	GBR	0.6354	0.8230	0.1674
	RFR	0.7583	0.9984	0.1963
	XGB	0.6407	0.8325	0.1687
	Wang et al.	0.0752	0.2797	0.0973
	Ours	**0.0167**	**0.0442**	**0.0037**

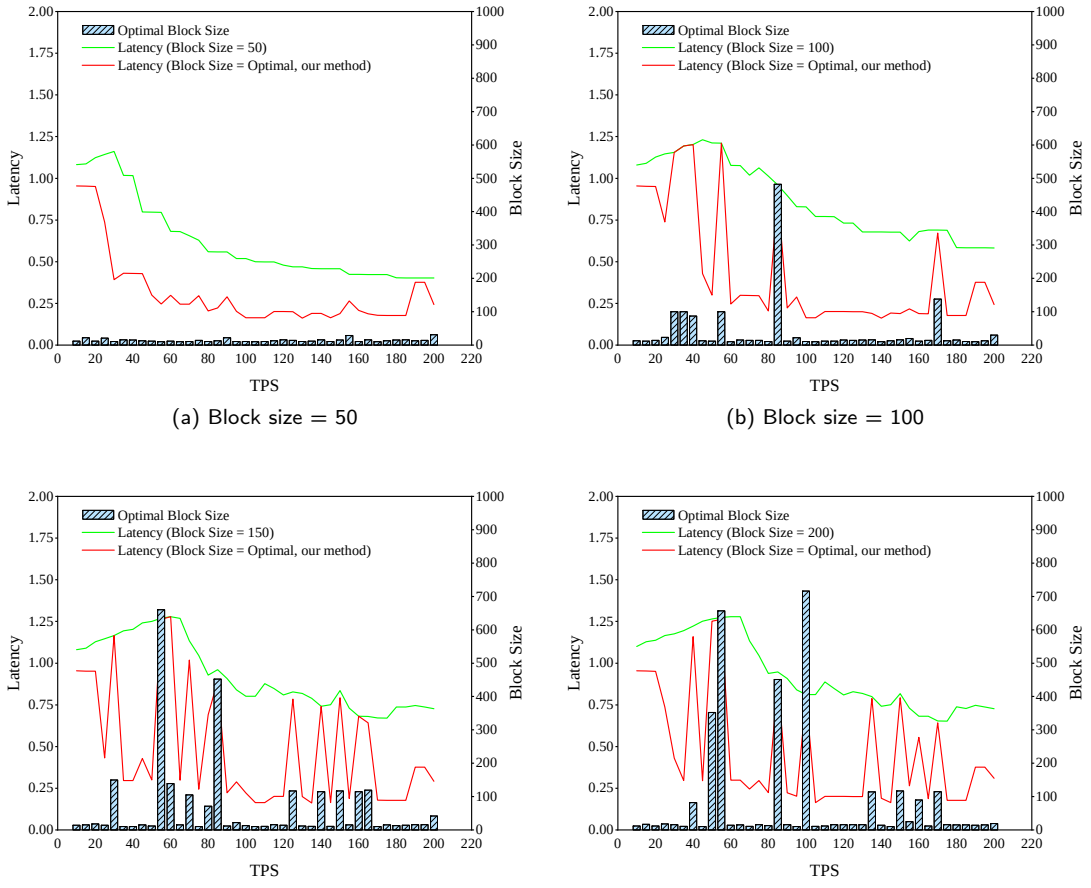
the effectiveness of the dynamic weighting mechanism and ensemble design in capturing complex latency behaviors.

Throughput prediction For throughput prediction, the proposed model achieved superior performance, especially at larger block sizes (150 and 200), where it accurately captured the nonlinear saturation behavior. The model achieved an MAE of 0.0167 and a MAPE of 0.0037, improving over both individual regressors and the prior blockchain-oriented prediction baseline, as shown in Table 2.

5.2.3. *Performance of the optimization mechanism with DW-DQN*

This section evaluates the proposed blockchain performance optimization mechanism based on the Dynamic Weight Deep Q-Network (DW-DQN), focusing on improvements in system latency and throughput. Experiments were conducted with four initial block sizes (50, 100, 150, and 200) under various transaction loads (TPS). The results demonstrate DW-DQN's adaptability and robustness in solving multi-objective optimization tasks.

DW-DQN consistently enhances system performance across various TPS levels, with notable improvements observed under high-load conditions. For example, when the initial block size was set to 150 and TPS increased from 60 to 140, the latency before optimization reached approximately 1.25 seconds. In contrast, DW-DQN maintained latency between 0.25 and 0.9 seconds, achieving an average improvement of 54% (Figure 7).



(a) Block size = 50 (b) Block size = 100

(c) Block size = 150 (d) Block size = 200

Figure 7: Latency optimization under different TPS and block sizes using DW-DQN.

Table 3
The optimization rate of optimal block size on latency

Metric	Block Size				
	50	100	150	200	Optimal (Ours)
Average Latency (s)	0.62	0.87	0.93	0.93	**0.43**
Optimization Rate (%)	49.78	50.73	47.36	50.09	—

Table 4
The optimization rate of optimal block size on throughput

Metric	Block Size				
	50	100	150	200	Optimal (Ours)
Average Throughput (TPS)	84.74	85.74	92.60	92.37	**103.09**
Optimization Rate (%)	22.09	20.70	10.66	11.35	—

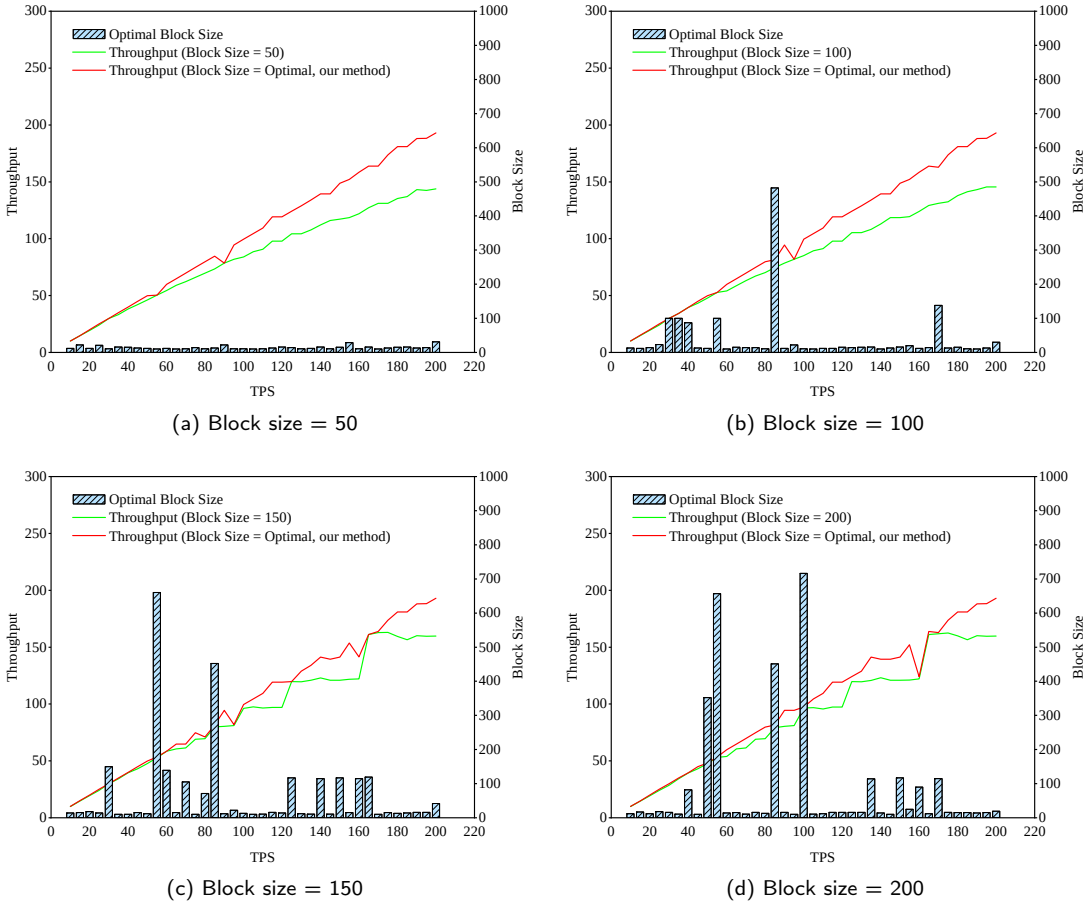
In terms of throughput, DW-DQN outperforms fixed block size baselines, especially under high TPS conditions. For example, at TPS = 160, DW-DQN achieved a maximum throughput 30% higher than the static approach (Figure 8).

Tables 3 and 4 summarize the optimization rates for latency and throughput, respectively. DW-DQN significantly reduced average latency and increased throughput across all initial block sizes. For example, with an initial block size of 200, latency was reduced from 0.93 seconds to 0.43 seconds (50.09% improvement), and throughput increased from 92.37 TPS to 103.09 TPS (11.35% improvement).

5.3. Ablation study

To evaluate the contribution of individual components within ChainOptNet, we conducted an ablation study on the HFBDP dataset. The complete optimization pipeline was retained, while one key component was removed or replaced at a time. Specifically, the CFAD module was replaced with a standard LSTM predictor, the ensemble prediction framework was replaced with the best-performing individual predictor, and the dynamic weighting mechanism was replaced with a fixed-weight reward design. The optimization performance was evaluated using average latency and average throughput over all tested workload scenarios.

To ensure a fair comparison, all ablation variants were evaluated under the same HFBDP workload traces, blockchain configurations, and optimization settings. The reported results correspond to the average performance across all tested transaction arrival rates (TPS) and initial block-size settings.



(a) Block size = 50 (b) Block size = 100

(c) Block size = 150 (d) Block size = 200

Figure 8: Throughput optimization under different TPS and block sizes using DW-DQN.

Table 5
Ablation study results on the HFBDP dataset

Model	Average Latency (s)	Average Throughput (TPS)
Full ChainOptNet	**0.43**	**103.09**
w/o CFAD	0.49	98.37
w/o Ensemble Prediction	0.47	99.82
w/o Dynamic Weighting	0.58	93.74

As shown in Table 5, removing the CFAD module leads to noticeable performance degradation. The average latency increases from 0.43 s to 0.49 s, while the average throughput decreases from 103.09 TPS to 98.37 TPS. This result indicates that accurate transaction arrival rate prediction is essential for effective block-size optimization.

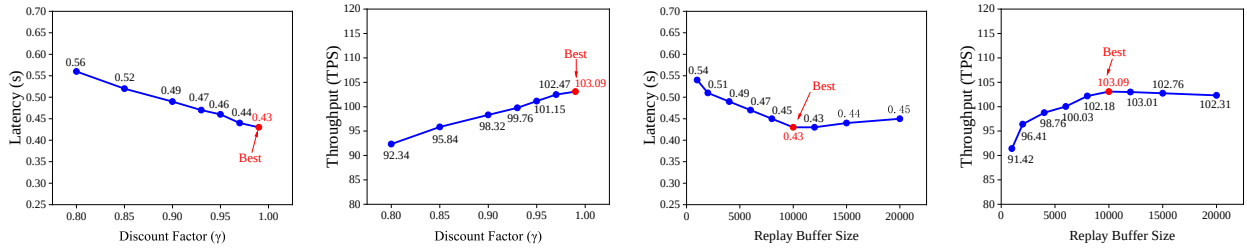
Replacing the ensemble prediction framework with a single predictor also reduces optimization performance. Although the degradation is less severe than removing CFAD, the increase in latency and decrease in throughput demonstrate the importance of combining heterogeneous prediction models. The inverse-error-weighted ensemble strategy improves prediction robustness and provides more reliable performance estimates for the optimization module.

The most significant performance degradation is observed when the dynamic weighting mechanism is removed from DW-DQN. In this case, the average latency increases to 0.58 s and the average throughput drops to 93.74 TPS. This finding confirms that adaptive reward balancing plays a critical role in maintaining an effective trade-off between latency and throughput under varying workload conditions.

Overall, the results validate the necessity and complementary effectiveness of the three major components in ChainOptNet. CFAD provides accurate workload forecasting, the ensemble prediction framework improves blockchain performance estimation, and the dynamic weighting mechanism enables adaptive multi-objective optimization. Together, these components contribute to the superior performance achieved by the proposed framework.

5.4. Hyperparameter analysis

To investigate the sensitivity of the proposed optimization framework to key DW-DQN hyperparameters, we conducted a comprehensive sensitivity analysis by varying the discount factor and replay buffer size on the HFBDP dataset. The corresponding latency and throughput obtained under different parameter settings are summarized in Table 6, while the variation trends are illustrated in Fig. 9.



(a) Latency vs. Discount Factor (b) Throughput vs. Discount Factor (c) Latency vs. Replay Buffer Size (d) Throughput vs. Replay Buffer Size

Figure 9: Sensitivity analysis of DW-DQN hyperparameters. (a) and (b) show the impact of discount factor on latency and throughput, respectively. (c) and (d) show the impact of replay buffer size on latency and throughput, respectively.

Table 6
Sensitivity analysis of DW-DQN hyperparameters.

Parameter	Value	Latency (s)	Throughput (TPS)
γ	0.80	0.56	92.34
γ	0.85	0.52	95.84
γ	0.90	0.49	98.32
γ	0.93	0.47	99.76
γ	0.95	0.46	101.15
γ	0.97	0.44	102.47
γ	**0.99**	**0.43**	**103.09**
Replay Buffer	1000	0.54	94.12
Replay Buffer	2000	0.51	96.41
Replay Buffer	4000	0.49	98.76
Replay Buffer	6000	0.47	100.03
Replay Buffer	8000	0.45	102.18
Replay Buffer	**10000**	**0.43**	**103.09**
Replay Buffer	12000	0.43	103.01
Replay Buffer	15000	0.44	102.76
Replay Buffer	20000	0.45	102.31

Discount factor (γ). The discount factor determines the importance of future rewards during policy learning. As shown in Table 6, the optimization performance improves steadily as γ increases from 0.80 to 0.99. Specifically, the average latency decreases from 0.56 s to 0.43 s, while the average throughput increases from 92.34 TPS to 103.09 TPS. When γ is relatively small, the agent mainly focuses on immediate rewards, resulting in conservative block-size adjustment decisions and relatively lower optimization performance. As γ increases beyond 0.93, the agent gradually learns more effective long-term optimization policies, leading to noticeable reductions in latency and continuous improvements in throughput. The best optimization performance is achieved when $\gamma = 0.99$, indicating that emphasizing long-term cumulative rewards is beneficial for dynamic blockchain parameter optimization.

Replay buffer size. The replay buffer size controls the number of historical experiences used for network updates. As the replay buffer size increases from 1000 to 10000, both latency and throughput are continuously improved. In particular, the average latency decreases from 0.54 s to 0.43 s, while the throughput increases from 94.12 TPS to 103.09 TPS. This improvement demonstrates that a larger replay buffer provides more diverse training samples, thereby improving policy stability and convergence. However, when the replay buffer size exceeds 10000, the performance gain becomes marginal. Although the replay buffer continues to increase to 12000, 15000, and 20000, the throughput remains almost unchanged while the latency exhibits a slight increase. This observation suggests that excessively large replay memories introduce additional storage overhead without providing significant improvements in optimization performance.

Overall, the proposed DW-DQN framework demonstrates stable optimization performance under different hyperparameter settings. Both the discount factor and replay buffer size exhibit clear convergence trends, indicating that the proposed optimization strategy is relatively robust to hyperparameter variations. Based on the above analysis, a discount factor of 0.99 and a replay buffer size of 10000 are selected for all subsequent experiments, as they provide the best trade-off between optimization effectiveness and computational efficiency.

5.5. Comparative analysis

Figure 10 shows that the proposed DW-DQN method consistently outperforms representative prediction-driven, score-based optimization methods across varying transaction loads, particularly under high-TPS conditions. As TPS increases, the latency reduction achieved by this class of baselines gradually plateaus, whereas DW-DQN continues to reduce latency significantly. For example, at TPS = 200, DW-DQN achieves a substantial latency reduction compared with the reference score-based strategy, demonstrating superior adaptability to performance degradation under heavy workloads.

As the transaction load increases, the performance gap between DW-DQN and the score-based baselines becomes increasingly evident. Under low-load conditions, both types of methods achieve relatively low latency since the blockchain network is not heavily congested. However, as TPS exceeds 120, score-based strategies gradually lose their ability to effectively adapt to rapidly changing workloads, resulting

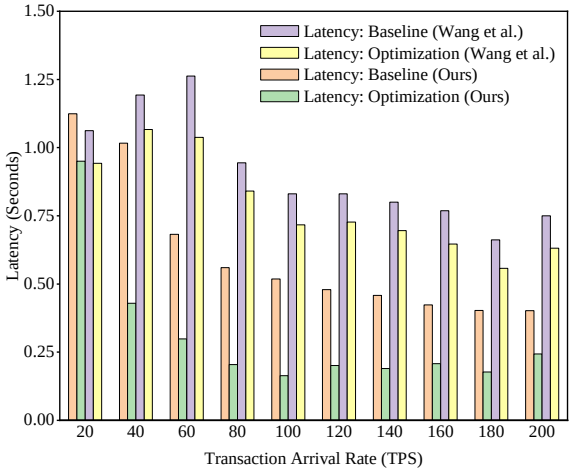


Figure 10: Latency comparison under different TPS.

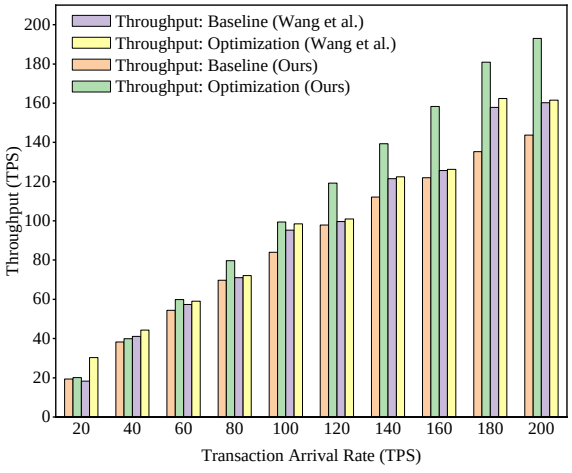


Figure 11: Throughput comparison under different TPS.

in a noticeable increase in latency. In contrast, DW-DQN maintains consistently lower latency by continuously adjusting the block size according to the current network state, demonstrating stronger adaptability under dynamic blockchain environments.

Figure 11 further illustrates that DW-DQN consistently achieves higher throughput than the score-based baselines across all tested TPS levels, with particularly noticeable improvements under medium- and high-load scenarios (TPS $>$ 100). For instance, at TPS $=$ 180 and TPS $=$ 200, DW-DQN improves throughput by approximately 25 TPS compared with the reference score-based strategy, demonstrating better scalability in transaction-intensive environments.

A similar trend can also be observed in terms of throughput. As the transaction arrival rate increases, DW-DQN maintains stable throughput growth while score-based strategies exhibit performance saturation. This indicates that the proposed optimization strategy can better exploit the available network resources by dynamically selecting appropriate block sizes. Consequently, the blockchain system is able to process more transactions without introducing excessive communication or confirmation delays.

The superior performance can be attributed to DW-DQN's ability to effectively model the nonlinear relationship among transaction load, block size, and system performance. Furthermore, the incorporation of experience replay and a discrete action space enhances training stability and facilitates smoother policy convergence, enabling the learned policy to generalize well under dynamic blockchain workloads. Overall, the experimental results demonstrate that the proposed optimization framework not only improves latency and throughput simultaneously, but also exhibits strong scalability and robustness under diverse transaction workloads, making it well suited for practical blockchain deployment scenarios.

6. Conclusion

This paper introduces ChainOptNet, a comprehensive blockchain performance optimization framework that achieves significant system improvements by leveraging three key technical innovations. First, for transaction arrival rate prediction, we introduce the Convolutional Feature Aggregation and Distribution (CFAD) model, which leverages a novel deep learning architecture to effectively capture temporal dependencies and complex distributional patterns in transaction data. Second, in the domain of performance prediction, a multi-model collaborative framework is proposed to improve predictive accuracy and robustness by intelligently combining heterogeneous machine learning models. Third, the Dynamic Weight Deep Q-Network (DW-DQN) serves as the optimization mechanism, combining a DQN-based block-size controller with a trend-aware dynamic reward weighting module to support real-time adaptive block-size adjustment and balanced optimization across multiple objectives.

Extensive experimental evaluations demonstrate the effectiveness of the proposed framework. The results show that ChainOptNet improves latency reduction and throughput enhancement compared with representative forecasting, regression, ensemble-learning, fixed-configuration, and score-based optimization baselines, while maintaining high optimization efficiency across varying transaction loads. The framework has a high degree of generalization and adaptability, making it suitable for use in a wide range of dynamic blockchain environments.

Future research will investigate several promising directions to improve ChainOptNet's applicability and performance. These include: (1) lightweight optimization algorithms to reduce computational overhead, (2) extending the framework to support cross-chain performance optimization in heterogeneous public chain ecosystems, (3) designing scenario-specific optimization strategies tailored to particular application domains, (4) incorporating advanced security and privacy mechanisms during optimization processes, and (5) improving system adaptability. These future efforts are

intended to accelerate the practical deployment of high-performance, intelligent blockchain infrastructure.

CRediT authorship contribution statement

Author 1: Conceptualization, Methodology, Writing - final draft. Author 2: carrying out experiments and Writing - original draft. The remaining authors contributed to validating the ideas, carrying out additional analyses and reviewing this paper. All authors read and approved the manuscript.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the ...

References

[1] Lingyan Xue, Haiping Huang, Fu Xiao, Qi Li, and Zhiwei Wang. A privacy-enhanced traceable anonymous transaction scheme for blockchain. *IEEE Transactions on Information Forensics and Security*, 20:1176–1191, 2025.

[2] Oluwafemi Akanfe, Diane Lawong, and H Raghav Rao. Blockchain technology and privacy regulation: Reviewing frictions and synthesizing opportunities. *International Journal of Information Management*, 76:102753, 2024.

[3] Wanqing Jie, Wangjie Qiu, Arthur Sandor Voundi Koe, Jianhong Li, Yin Wang, Yaqi Wu, Jin Li, and Zhiming Zheng. A secure and flexible blockchain-based offline payment protocol. *IEEE Transactions on Computers*, 73(2):408–421, 2023.

[4] Li Lin, Jiapeng Wu, Zhi Zhou, Jin Zhao, Peng Li, and Jinbo Xiong. Computing power networking meets blockchain: A reputation-enhanced trading framework for decentralized iot cloud services. *IEEE Internet of Things Journal*, 11(10):17082–17096, 2024.

[5] Yuming Huang, Xiangyu Li, Yuehan Wu, Chaoyang Xue, Jiashuo Li, Yongfeng Lin, Wei Nie, Xian Liu, Qian Liu, Greg Michalski, et al. Blockchain-based isotopic big data-driven tracing of global pm sources and interventions. *Nature Communications*, 16(1):3901, 2025.

[6] Di Zhang, Junqing Le, Xinyu Lei, Tao Xiang, and Xiaofeng Liao. Secure redactable blockchain with dynamic support. *IEEE Transactions on Dependable and Secure Computing*, 21(2):717–731, 2023.

[7] Jie Xu, Cong Wang, and Xiaohua Jia. A survey of blockchain consensus protocols. *ACM Computing Surveys*, 55(13s):1–35, 2023.

[8] QingQiu Gan and Raymond Yiu Keung Lau. Trust in a 'trust-free'system: Blockchain acceptance in the banking and finance sector. *Technological forecasting and social change*, 199:123050, 2024.

[9] Mohd Javaid, Abid Haleem, Ravi Pratap Singh, Rajiv Suman, and Shahbaz Khan. A review of blockchain technology applications for financial services. *BenchCouncil transactions on benchmarks, standards and evaluations*, 2(3):100073, 2022.

[10] Ruibiao Chen, Fangxing Shu, Shuokang Huang, Lei Huang, Huafang Liu, Jin Liu, and Kai Lei. Bidm: A blockchain-enabled cross-domain identity management system. *Journal of Communications and Information Networks*, 6(1):44–58, 2021.

[11] Jie Cui, Yatao Li, Qingyang Zhang, Hong Zhong, Chengjie Gu, and Debiao He. Dschain: A blockchain system for complete lifecycle security of data in internet of things. *IEEE Transactions on Dependable and Secure Computing*, 21(4):3977–3993, 2023.

[12] Sadia Ramzan, Aqsa Aqdus, Vinayakumar Ravi, Deepika Koundal, Rashid Amin, and Mohammed A Al Ghamdi. Healthcare applications using blockchain technology: Motivations and challenges. *IEEE Transactions on Engineering Management*, 70(8):2874–2890, 2022.

[13] Keke Gai, Yue Zhang, Meikang Qiu, and Bhavani Thuraisingham. Blockchain-enabled service optimizations in supply chain digital twin. *IEEE Transactions on Services Computing*, 16(3):1673–1685, 2022.

[14] Keke Gai, Yulu Wu, Liehuang Zhu, Meikang Qiu, and Meng Shen. Privacy-preserving energy trading using consortium blockchain in smart grid. *IEEE Transactions on Industrial Informatics*, 15(6):3548–3558, 2019.

[15] Junqin Huang, Linghe Kong, Long Cheng, Hong-Ning Dai, Meikang Qiu, Guihai Chen, Xue Liu, and Gang Huang. Blocksense: Towards trustworthy mobile crowdsensing via proof-of-data blockchain. *IEEE Transactions on Mobile Computing*, 23(2):1016–1033, 2022.

[16] Ammar Ibrahim El Sayed, Mahmoud Abdelaziz, Mohamed Hussein, and Ashraf D Elbayoumy. Ddos mitigation in iot using machine learning and blockchain integration. *IEEE Networking Letters*, 2024.

[17] Bakkiam David Deebak, Fida Hussain Memon, Sunder Ali Khowaja, Kapal Dev, Weizheng Wang, Nawab Muhammad Faseeh Qureshi, and Chunhua Su. A lightweight blockchain-based remote mutual authentication for ai-empowered iot sustainable computing systems. *IEEE Internet of Things Journal*, 10(8):6652–6660, 2022.

[18] Shen Wang, Ahmad F Taha, Jianhui Wang, Karla Kvaternik, and Adam Hahn. Energy crowdsourcing and peer-to-peer energy trading in blockchain-enabled smart grids. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 49(8):1612–1623, 2019.

[19] Zhenyu Zhou, Bingchen Wang, Mianxiong Dong, and Kaoru Ota. Secure and efficient vehicle-to-grid energy trading in cyber physical systems: Integration of blockchain and edge computing. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 50(1):43–57, 2019.

[20] Tran Viet Khoa, Do Hai Son, Dinh Thai Hoang, Nguyen Linh Trung, Tran Thi Thuy Quynh, Diep N Nguyen, Nguyen Viet Ha, and Eryk Dutkiewicz. Collaborative learning for cyberattack detection in blockchain networks. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 54(7):3920–3933, 2024.

[21] Gi Seok Park and Hwangjun Song. Watchdog: Network-aware consensus protocol for enhancing scalability of public blockchains. *IEEE Internet of Things Journal*, 2024.

[22] Jemin Andrew Choi, Sidi Mohamed Beillahi, Srisht Fateh Singh, Panagiotis Michalopoulos, Peilun Li, Andreas Veneris, and Fan Long. Lmpt: A novel authenticated data structure to eliminate storage bottlenecks for high performance blockchains. *IEEE Transactions on Network and Service Management*, 21(2):1333–1343, 2023.

[23] Wei Liu, Zhiguo Wan, Jun Shao, and Yong Yu. Hypermaze: Towards privacy-preserving and scalable permissioned blockchain. *IEEE Transactions on Dependable and Secure Computing*, 20(1):360–376, 2021.

[24] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.

[25] Vitalik Buterin et al. Ethereum white paper. *GitHub repository*, 1 (22-23):5–7, 2013.

[26] Aklima Begum, Mosa Sumaiya Khatun Munira, and Shaharima Juthi. Systematic review of blockchain technology in trade finance and banking security. *American Journal of Scholarly Research and Innovation*, 1(01):25–52, 2022.

[27] Qinyin Ni, Zhang Linfeng, Xiaorong Zhu, and Inayat Ali. A novel design method of high throughput blockchain for 6g networks: Performance analysis and optimization model. *IEEE Internet of Things Journal*, 9(24):25643–25659, 2022.

[28] Min An, Xuan Zhang, Jishu Wang, Qiyuan Fan, Chen Gao, Linyu Li, Cuizhen Lu, Nan Li, and Yingchen Liu. Rlchain: A drl approach for

blockchain performance optimization towards iiot. *IEEE Transactions on Network and Service Management*, 2024.

[29] Jianing Chen, Jun Wu, Haoran Liang, Shahid Mumtaz, Jianhua Li, Kostromitin Konstantin, Ali Kashif Bashir, and Raheel Nawaz. Collaborative trust blockchain based unbiased control transfer mechanism for industrial automation. *IEEE Transactions on Industry Applications*, 56(4):4478–4488, 2019.

[30] Yuhan Yang, Jing Wu, Chengnian Long, Wei Liang, and Yi-Bing Lin. Blockchain-enabled multiparty computation for privacy preserving and public audit in industrial iot. *IEEE Transactions on Industrial Informatics*, 18(12):9259–9267, 2022.

[31] Guoqiong Liao, Hao Ding, Chuanling Zhong, and Yinxiang Lei. Rt-dag: A dag-based blockchain supporting real-time transactions. *IEEE Internet of Things Journal*, 2024.

[32] Jie Xu, Qingyuan Xie, Sen Peng, Cong Wang, and Xiaohua Jia. Adaptchain: Adaptive scaling blockchain with transaction deduplication. *IEEE Transactions on Parallel and Distributed Systems*, 34(6): 1909–1922, 2023.

[33] Ningjie Gao, Ru Huo, Shuo Wang, Tao Huang, and Yunjie Liu. Sharding-hashgraph: A high-performance blockchain-based framework for industrial internet of things with hashgraph mechanism. *IEEE Internet of Things Journal*, 9(18):17070–17079, 2021.

[34] Weishan Zhang, Gang Sun, Liang Xu, Qinghua Lu, Huansheng Ning, Peiying Zhang, and Su Yang. A trustworthy safety inspection framework using performance-security balanced blockchain. *IEEE Internet of Things Journal*, 9(11):8178–8190, 2021.

[35] Xiaohai Dai, Zhaonan Zhang, Zhengxuan Guo, Chaozheng Ding, Jiang Xiao, Xia Xie, Rui Hao, and Hai Jin. Wahoo: A dag-based bft consensus with low latency and low communication overhead. *IEEE Transactions on Information Forensics and Security*, 2024.

[36] Jinmian Chen, Yukun Cheng, Zhiqi Xu, and Yan Cao. Decision on block size in blockchain systems by evolutionary equilibrium analysis. *Theoretical Computer Science*, 942:93–106, 2023.

[37] Maruf Monem, Md Tamjid Hossain, Md Golam Rabiul Alam, Md Shirajum Munir, Md Mahbubur Rahman, Salman A AlQahtani, Samah Almutlaq, and Mohammad Mehedi Hassan. A sustainable bitcoin blockchain network through introducing dynamic block size adjustment using predictive analytics. *Future Generation Computer Systems*, 153:12–26, 2024.

[38] Faisal Jamil, Shabir Ahmad, Taeg Keun Whangbo, Ammar Muthanna, and Do-Hyeun Kim. Improving blockchain performance in clinical trials using intelligent optimal transaction traffic control mechanism in smart healthcare applications. *Computers & Industrial Engineering*, 170:108327, 2022.

[39] Xuan Chen, Kien Nguyen, and Hiroo Sekiya. On the latency performance in private blockchain networks. *IEEE Internet of Things Journal*, 9(19):19246–19259, 2022.

[40] Francesc Wilhelmi, Sergio Barrachina-Muñoz, and Paolo Dini. End-to-end latency analysis and optimal block size of proof-of-work blockchain applications. *IEEE Communications Letters*, 26(10): 2332–2335, 2022.

[41] Jishu Wang, Chao Zhu, Chen Miao, Rui Zhu, Xuan Zhang, Yahui Tang, Hexiang Huang, and Chen Gao. Bpr: Blockchain-enabled efficient and secure parking reservation framework with block size dynamic adjustment method. *IEEE Transactions on Intelligent Transportation Systems*, 24(3):3555–3570, 2022.

[42] Jishu Wang, Yaowei Wang, Xuan Zhang, Zhi Jin, Chao Zhu, Linyu Li, Rui Zhu, and Shenglong Lv. Learningchain: A highly scalable and applicable learning-based blockchain performance optimization framework. *IEEE Transactions on Network and Service Management*, 21(2):1817–1831, 2023.

[43] Mingzhe Li, Wei Wang, and Jin Zhang. Lb-chain: Load-balanced and low-latency blockchain sharding via account migration. *IEEE Transactions on Parallel and Distributed Systems*, 34(10):2797–2810, 2023.

[44] Meiqi Li, Xinyi Luo, Kaiping Xue, Yingjie Xue, Wentuo Sun, and Jian Li. A secure and efficient blockchain sharding scheme via hybrid consensus and dynamic management. *IEEE Transactions on Information Forensics and Security*, 2024.

[45] Md Mainul Islam, Mpyana Mwamba Merlec, and Hoh Peter In. Proof of random leader: A fast and manipulation-resistant proof-of-authority consensus algorithm for permissioned blockchains using verifiable random function. *IEEE Transactions on Services Computing*, 2025.

[46] Zhongteng Cai, Junyuan Liang, Wuhui Chen, Zicong Hong, Hong-Ning Dai, Jianting Zhang, and Zibin Zheng. Benzene: Scaling blockchain with cooperation-based sharding. *IEEE Transactions on Parallel and Distributed Systems*, 34(2):639–654, 2022.

[47] Wei Yang, Weilin Chen, Lide Xue, Wenjie Zou, and Liusheng Huang. Ezchain: A secure scalable blockchain protocol via passive sharding. *IEEE Transactions on Dependable and Secure Computing*, 2024.

[48] Yizhong Liu, Xinxin Xing, Haosu Cheng, Dawei Li, Zhenyu Guan, Jianwei Liu, and Qianhong Wu. A flexible sharding blockchain protocol based on cross-shard byzantine fault tolerance. *IEEE Transactions on Information Forensics and Security*, 18:2276–2291, 2023.

[49] Linpeng Jia, Yanxiu Liu, Keyuan Wang, and Yi Sun. Estuary: A low cross-shard blockchain sharding protocol based on state splitting. *IEEE Transactions on Parallel and Distributed Systems*, 35(3):405–420, 2024.

[50] Jie Xu, Yulong Ming, Zihan Wu, Cong Wang, and Xiaohua Jia. X-shard: Optimistic cross-shard transaction processing for sharding-based blockchains. *IEEE Transactions on Parallel and Distributed Systems*, 35(4):548–559, 2024.

[51] Jiewu Leng, Douxi Yan, Qiang Liu, Kailin Xu, J Leon Zhao, Rui Shi, Lijun Wei, Ding Zhang, and Xin Chen. Manuchain: Combining permissioned blockchain with a holistic optimization model as bi-level intelligence for smart manufacturing. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 50(1):182–192, 2019.

[52] Alexander Marsalek, Thomas Zefferer, Edona Fasllija, and Dominik Ziegler. Tackling data inefficiency: Compressing the bitcoin blockchain. In *2019 18th IEEE international conference on trust, security and privacy in computing and communications/13th IEEE international conference on big data science and engineering (trustcom/bigdatase)*, pages 626–633. IEEE, 2019.

[53] Aolin Xu and Maxim Raginsky. Information-theoretic analysis of generalization capability of learning algorithms. *Advances in neural information processing systems*, 30, 2017.

[54] Lu Han, Xu-Yang Chen, Han-Jia Ye, and De-Chuan Zhan. Softs: Efficient multivariate time series forecasting with series-core fusion. *arXiv preprint arXiv:2404.14197*, 2024.